DATA STRUCTURES

CO1 Notes: Arrays, Strings and Linked Lists (Unit I)

Course Code: PBC2TE302 | PBCA | Sem III

Course Outcome (CO1)

To write neat code by selecting appropriate data structure. Gain proficiency in implementing and manipulating various types of linked lists and applying them to real-world problems.

Topics covered in these notes

- Introduction to data structures, types of data structures
- Pointers and dynamic memory
- Arrays: memory representation and operations (traversal, insertion, deletion)
- Strings and string operations
- Linked lists: single, double, header, circular single and circular double, memory representation
- All operations on linked lists
- Application: polynomial representation and manipulation
- Quick revision and practice questions

1. Introduction to Data Structures

Data means raw facts and figures (for example marks, names, numbers). A **Data Structure** is a way of **storing, organising and managing data in the computer memory** so that it can be used efficiently. Think of a data structure like a cupboard: books can be thrown in randomly, but if you arrange them on shelves by subject you can find any book quickly. In the same way, a good data structure makes *searching, inserting, deleting and sorting* data fast.

Why do we need data structures?

- To store large amounts of data in an organised way.
- To make operations like search, insert, delete and sort fast.
- To use computer memory and time efficiently.
- To write neat, reusable and easy-to-understand programs.

Classification (Types) of Data Structures

Data structures are mainly divided into **Primitive** and **Non-primitive** data structures.

Type	Meaning	Examples
Primitive	Basic data types that are directly supported by the machine / language.	int, float, char, double, pointer
Non-primitive (Linear)	Elements are stored one after another in a **sequence**. Every element has one previous and one next element (except first and last).	Array, Linked List, Stack, Queue
Non-primitive (Non-linear)	Elements are NOT in a sequence. One element can be connected to many others (hierarchy or network).	Tree, Graph

Other ways of classifying

- **Static data structure:** size is fixed at compile time (example: array).
- **Dynamic data structure:** size can grow or shrink while the program runs (example: linked list).

- **Homogeneous:** all elements are of the same type (array). **Heterogeneous:** elements may be of different types (structure).

Basic operations on data structures

Operation	Meaning
Traversal	Visiting every element exactly once (for example to print all).
Insertion	Adding a new element.
Deletion	Removing an existing element.
Searching	Finding whether an element is present and where.
Sorting	Arranging elements in ascending or descending order.
Merging	Combining two data structures into one.

Abstract Data Type (ADT)

An ADT tells *what* operations can be done on the data, not *how* they are done. For example, a Stack ADT says "push, pop, peek" are possible; whether it is built using an array or a linked list is hidden.

2. Pointers

A **pointer** is a variable that stores the **memory address** of another variable. Pointers are the backbone of linked lists, trees and graphs.

- **& (address-of operator)** gives the address of a variable.
- *** (dereference operator)** gives the value stored at that address.
- **NULL pointer** is a pointer that points to nothing (address 0). It is used to mark the end of a list.

```
int a = 10;
int *p;          // p is a pointer to an integer
p = &a;          // p now holds the address of a
printf("%d", a);   // 10
printf("%p", p);   // address of a  (for example 1000)
printf("%d", *p);  // 10  (value at the address stored in p)
*p = 25;         // changes the value of a to 25
```

Variable	Address	Value
a	1000	10
p	2000	1000 (address of a)

Dynamic memory allocation

Normal variables get fixed memory. With pointers we can ask for memory **at run time**:

- **malloc(size)** - allocates a block of memory and returns its address.
- **calloc(n,size)** - allocates memory for n items and sets them to 0.
- **realloc(ptr,size)** - changes the size of an already allocated block.
- **free(ptr)** - releases memory so that it can be used again (avoids memory leak).

```
struct node *newnode;
newnode = (struct node *) malloc(sizeof(struct node));  // create a node at run time
free(newnode);                                          // delete it when not needed
```

3. Arrays

3.1 Definition

An **array** is a collection of **similar (same data type) elements** stored in **continuous (contiguous) memory locations** and accessed using a common name and an **index** (subscript). The index of the first element is **0** in C.

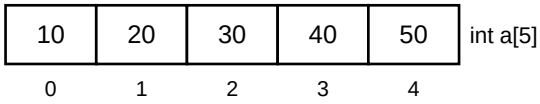


Fig 1.1: One-dimensional array of 5 integers (index numbers are written below the boxes)

Features

- Elements are stored one after the other in memory.
- Any element can be accessed directly using its index (random access) in constant time.
- Size is fixed when the array is declared (static).

Declaration and initialisation

```
int a[5];                        // declaration
int b[5] = {10, 20, 30, 40, 50};  // declaration with initialisation
int c[]  = {1, 2, 3};             // size is decided by the compiler (3)
int m[2][3] = {{1,2,3},{4,5,6}};  // two-dimensional array (2 rows, 3 columns)
```

3.2 Memory representation

Because elements are stored continuously, the address of any element can be found with a formula:

Address formula (1-D array)

- **Address of A[i] = Base address + i x size of one element**
- Example: base address = 1000, int takes 4 bytes. Address of a[3] = 1000 + 3 x 4 = **1012**.

Index	a[0]	a[1]	a[2]	a[3]	a[4]
Value	10	20	30	40	50
Address	1000	1004	1008	1012	1016

Two-dimensional arrays are stored either **row-major** (row by row, used in C) or **column-major** (column by column). Address of A[i][j] in row-major = Base + (i x number_of_columns + j) x size.

3.3 Operations on arrays

(a) Traversal - visit every element

```
for (i = 0; i < n; i++)
    printf("%d ", a[i]);
```

(b) Insertion - add an element at a given position

Because elements are in continuous memory, to insert at position *pos* we must first **shift all elements from pos to the end one place to the right** to make a gap. Then the new value is placed in the gap and the size n is increased by 1.

Algorithm: INSERT(A, n, pos, item)

Step 1: Start. Check that the array is not full (n < MAX).

Step 2: Set i = n - 1.

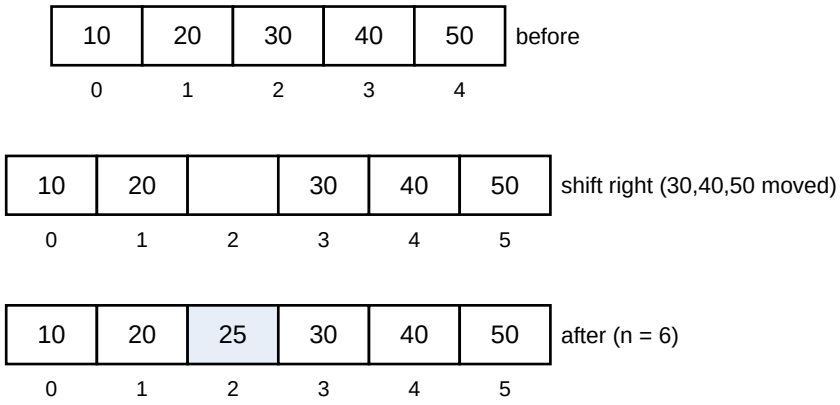
Step 3: Repeat while i >= pos : A[i+1] = A[i] ; i = i - 1.

Step 4: A[pos] = item.

Step 5: n = n + 1.

Step 6: Stop.

Example: Insert **25** at position 2 in the array [10, 20, 30, 40, 50] (positions start from 0).




```
#include <stdio.h>
int main() {
    int a[10] = {10, 20, 30, 40, 50}, n = 5, pos = 2, item = 25, i;
    for (i = n - 1; i >= pos; i--)   // shift elements to the right
        a[i + 1] = a[i];
    a[pos] = item;                   // place the new element
    n++;
    for (i = 0; i < n; i++) printf("%d ", a[i]);   // 10 20 25 30 40 50
    return 0;
}
```

(c) Deletion - remove the element at a given position

To delete, **shift all elements after the position one place to the left** so the gap is filled, then decrease n by 1.

Algorithm: DELETE(A, n, pos)

Step 1: Start. Check that the array is not empty (n > 0).

Step 2: item = A[pos].

Step 3: Set i = pos. Repeat while i < n - 1 : A[i] = A[i+1] ; i = i + 1.

Step 4: n = n - 1.

Step 5: Stop.

```
for (i = pos; i < n - 1; i++)
    a[i] = a[i + 1];     // shift left
n--;                     // [10,20,25,30,40,50] delete pos 2 -> [10,20,30,40,50]
```

(d) Searching and (e) Updating

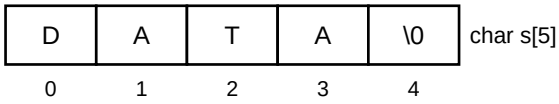
- **Searching:** compare each element with the key (linear search) - covered in CO4.
- **Updating:** a[index] = new_value; (direct and fast).

Advantages	Disadvantages
Fast direct access using index (random access).	Fixed size - cannot grow or shrink.

Advantages	Disadvantages
Easy to use and understand.	Insertion and deletion are slow (shifting needed).
Memory is allocated in one continuous block.	Wastes memory if size is declared bigger than needed.

4. Strings

A **string** is a collection of characters stored in a **character array** and ended with the special **null character '\0'**. For example "DATA" is stored as shown:




```
char s1[10] = "DATA";            // compiler adds '\0' automatically
char s2[]   = {'D','S','\0'};
scanf("%s", s1);                  // reads a word (stops at space)
gets(s1);  /  fgets(s1,10,stdin); // reads a line with spaces
printf("%s", s1);
```

Operations on strings

Operation	Library function (string.h)	Meaning
Length	strlen(s)	Counts characters before '\0'
Copy	strcpy(d, s)	Copies string s into d
Concatenation	strcat(s1, s2)	Joins s2 at the end of s1
Comparison	strcmp(s1, s2)	Returns 0 if equal, <0 if s1 < s2, >0 if s1 > s2
Reverse	strrev(s)	Reverses the string
Substring search	strstr(s, sub)	Finds the first position of sub in s

Program: length and reverse without library functions

```
char s[20] = "DATA", r[20];
int len = 0, i;
while (s[len] != '\0') len++;          // length = 4
for (i = 0; i < len; i++)
    r[i] = s[len - 1 - i];            // copy in reverse order
r[len] = '\0';
printf("%s", r);                       // ATAD
```

5. Linked Lists

5.1 Why linked list?

Arrays have a fixed size and insertion/deletion needs shifting. A **linked list** solves this. It is a **linear dynamic data structure** in which elements called **nodes** are stored at **different (non-continuous) memory locations** and connected with each other using **pointers (links)**.

Each node has two parts: **DATA** (the value) and **NEXT** (address of the next node). The first node is pointed to by **HEAD** (or START). The last node's next is **NULL**.

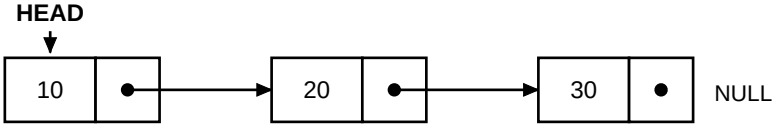


Fig 1.2: Singly linked list with three nodes

```
struct node {
    int data;
    struct node *next;     // pointer to the next node (self-referential structure)
};
struct node *head = NULL;  // empty list
```

5.2 Memory representation of a linked list

Nodes can be anywhere in memory. Only the links tell us the order. Suppose head = 200:

Address	DATA	NEXT
200 (first node)	10	450
450	20	120
120 (last node)	30	NULL (0)

So the list is: head(200) -> 10 -> 20 -> 30 -> NULL. The nodes are not in sorted address order but the links connect them in the correct order.

Array versus linked list

Array	Linked List
Continuous memory	Non-continuous memory
Fixed size (static)	Size grows/shrinks at run time (dynamic)
Direct access a[i]	Sequential access - must start from head
Insert/delete is slow (shifting)	Insert/delete is fast (only change links)
No extra memory for links	Extra memory needed for the pointer in each node

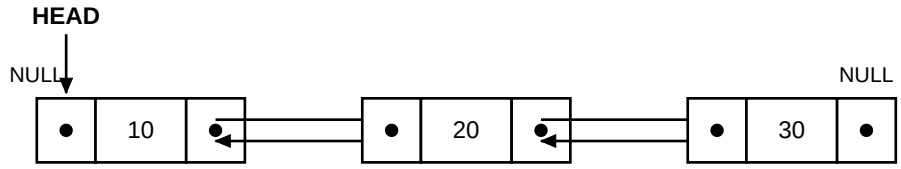
5.3 Types of linked lists

(a) Singly linked list

Each node has data and one pointer to the **next** node. Traversal is only in one direction (forward). Last node points to NULL.

(b) Doubly linked list

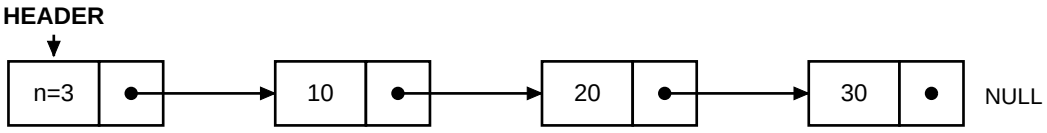
Each node has **three parts: PREV, DATA, NEXT**. We can move both forward and backward. Uses more memory but deletion is easier because we know the previous node.




```
struct dnode {
    struct dnode *prev;
    int data;
    struct dnode *next;
};
```

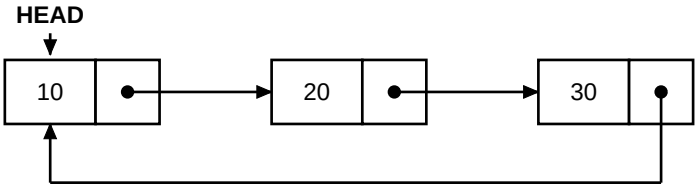
(c) Header linked list

A **header node** is a special extra node at the very beginning of the list. It does not store real data; it usually stores useful information like the **number of nodes** or a pointer to the last node. Because the header is always present, insertion at the beginning is treated like any other insertion (no special case for an empty list).



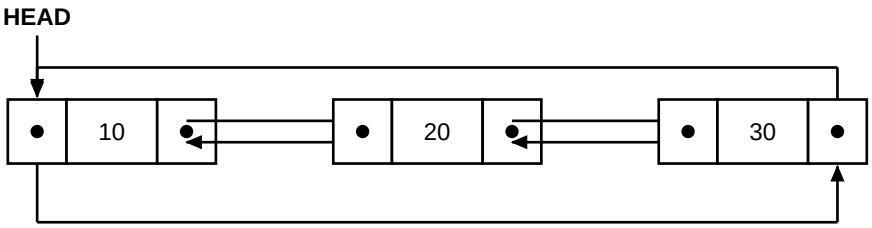
(d) Circular singly linked list

The next pointer of the **last node points back to the first node** instead of NULL. We can reach any node starting from any node. Used in round-robin scheduling, music playlists, etc.



(e) Circular doubly linked list

A doubly linked list where **last node's NEXT points to the first node** and **first node's PREV points to the last node**. We can move in both directions in a continuous circle.



Type	Links per node	Direction	Last node points to
Singly	1 (next)	Forward only	NULL
Doubly	2 (prev, next)	Forward and backward	NULL
Header	1 (+ header node)	Forward	NULL
Circular singly	1 (next)	Forward (circle)	First node
Circular doubly	2 (prev, next)	Both (circle)	First node (next) / last node (prev)

5.4 Operations on a singly linked list

(a) Creating a node

```
struct node *newnode = (struct node *) malloc(sizeof(struct node));
newnode->data = value;
newnode->next = NULL;
```

(b) Traversal (display)

Algorithm: TRAVERSE(head)

Step 1: temp = head.

Step 2: Repeat while temp != NULL : print temp->data ; temp = temp->next.

Step 3: Stop.

```
void display(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL");
}
```

(c) Insertion at the beginning

Idea: make the new node point to the old first node, then make head point to the new node.

Algorithm: INSERT_BEGIN(head, item)

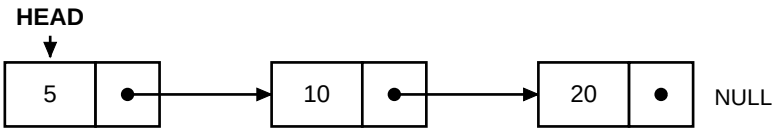
Step 1: Create a new node (newnode) using malloc. If memory is not available, print "Overflow" and stop.

Step 2: newnode->data = item.

Step 3: newnode->next = head.

Step 4: head = newnode.

Step 5: Stop.



After inserting 5 at the beginning of 10 -> 20 : new head is the node 5

```
struct node *insert_begin(struct node *head, int item) {
    struct node *newnode = (struct node *) malloc(sizeof(struct node));
    if (newnode == NULL) { printf("Overflow"); return head; }
    newnode->data = item;
    newnode->next = head;     // link new node to old first node
    head = newnode;           // new node becomes first node
    return head;
}
```

(d) Insertion at the end

Algorithm: INSERT_END(head, item)

Step 1: Create newnode, newnode->data = item, newnode->next = NULL.

Step 2: If head == NULL then head = newnode and stop.

Step 3: temp = head. Repeat while temp->next != NULL : temp = temp->next.

Step 4: temp->next = newnode.

Step 5: Stop.

```
temp = head;
while (temp->next != NULL) temp = temp->next;   // go to last node
temp->next = newnode;
```

(e) Insertion after a given position / node

Algorithm

Step 1: Create newnode with the data.

Step 2: Move temp to the node after which insertion is needed (loop pos-1 times).

Step 3: newnode->next = temp->next.

Step 4: temp->next = newnode. (Order is important: first Step 3, then Step 4)

(f) Deletion from the beginning

Idea: save the first node in a temporary pointer, move head to the second node, then free the saved node.

Algorithm: DELETE_BEGIN(head)

Step 1: If head == NULL, print "Underflow (list is empty)" and stop.

Step 2: temp = head.

Step 3: head = head->next.

Step 4: free(temp).

Step 5: Stop.

```
struct node *delete_begin(struct node *head) {
    struct node *temp;
    if (head == NULL) { printf("Underflow"); return head; }
    temp = head;           // remember first node
    head = head->next;     // second node becomes first
    free(temp);            // release memory
    return head;
}
```

Dry run: List = 10 -> 20 -> 30. temp points to 10, head moves to 20, node 10 is freed. New list = 20 -> 30.

(g) Deletion from the end

```
// if list has one node: free(head); head = NULL;
temp = head;
while (temp->next->next != NULL) temp = temp->next;  // reach second last node
free(temp->next);
temp->next = NULL;
```

(h) Deletion of a given node (by value)

Algorithm

Step 1: If the first node holds the key, delete it as "delete from beginning".

Step 2: Otherwise keep two pointers: prev and curr. Move both until curr->data == key (or curr is NULL).

Step 3: prev->next = curr->next.

Step 4: free(curr).

(i) Searching, counting and reversing

```
// search
for (temp = head; temp != NULL; temp = temp->next)
    if (temp->data == key) { printf("Found"); break; }
// count
int count = 0; for (temp = head; temp != NULL; temp = temp->next) count++;
// reverse (three pointers)
struct node *prev = NULL, *curr = head, *nxt;
while (curr != NULL) { nxt = curr->next; curr->next = prev; prev = curr; curr = nxt; }
head = prev;
```

5.5 Operations on a doubly linked list

Insert at beginning

newnode->prev = NULL ; newnode->next = head ;

if (head != NULL) head->prev = newnode ;

head = newnode ;

Insert at end

Go to the last node (temp). temp->next = newnode ; newnode->prev = temp ; newnode->next = NULL ;

Delete a node (pointed by temp)
temp->prev->next = temp->next ;
temp->next->prev = temp->prev ; (if temp is not the last node)
free(temp) ;

```
struct dnode *insert_begin(struct dnode *head, int item) {
    struct dnode *n = (struct dnode *) malloc(sizeof(struct dnode));
    n->data = item; n->prev = NULL; n->next = head;
    if (head != NULL) head->prev = n;
    return n;
}
```

5.6 Operations on circular linked lists

- **Traversal** must stop when we come back to the starting node (use do-while), not at NULL.
- **Insert at beginning:** create node, find last node, newnode->next = head, last->next = newnode, head = newnode.
- **Insert at end:** same as above but head is not changed.
- **Delete from beginning:** find last node, last->next = head->next, free(head), head = last->next.

```
temp = head;
do { printf("%d ", temp->data); temp = temp->next; } while (temp != head);
```

5.7 Advantages and disadvantages of linked lists

Advantages	Disadvantages
Dynamic size - grows and shrinks as needed.	Extra memory required for pointers.
Insertion and deletion are easy and fast.	No direct access - must traverse from head.
No memory wastage.	Traversal is slower than array; pointer handling is error-prone.

6. Application of Linked List: Polynomial Representation and Manipulation

A polynomial such as **$5x^3 + 4x^2 + 2x + 1$** can be stored in a linked list. Each term is one node with three fields: **coefficient, exponent** and **next**. Terms are kept in **decreasing order of exponent**. This is better than an array when the polynomial is sparse, for example $7x^{100} + 3$ needs only two nodes.

```
struct poly {
    int coeff;          // coefficient
    int exp;            // exponent (power of x)
    struct poly *next;
};
```

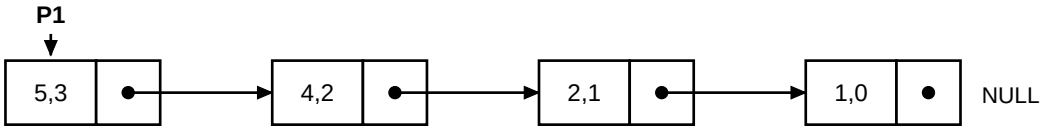


Fig 1.3: P1 = 5x^3 + 4x^2 + 2x + 1 stored as (coeff, exp) pairs

Polynomial addition

Use two pointers p1 and p2 (one on each polynomial) and compare exponents:

- If exp(p1) > exp(p2): copy term of p1 to the result and move p1.
- If exp(p1) < exp(p2): copy term of p2 to the result and move p2.
- If exponents are equal: add the coefficients, store the sum (if not zero) and move both pointers.
- When one polynomial ends, copy all remaining terms of the other.

Example: P1 = $5x^3 + 4x^2 + 2x + 1$ and P2 = $3x^2 + 6x + 7$

Step	p1 term	p2 term	Comparison	Result term
1	5x^3	3x^2	3 > 2	5x^3
2	4x^2	3x^2	2 = 2	(4+3) = 7x^2
3	2x	6x	1 = 1	(2+6) = 8x
4	1	7	0 = 0	(1+7) = 8

Result: P3 = $5x^3 + 7x^2 + 8x + 8$

Algorithm: ADD_POLY(P1, P2)

Step 1: p1 = P1, p2 = P2, result list R = empty.

Step 2: Repeat while p1 != NULL and p2 != NULL : compare exponents as per the rules above and append a node to R.

Step 3: Append the remaining nodes of P1 or P2 to R.

Step 4: Return R. Stop.

```
while (p1 != NULL && p2 != NULL) {
    if (p1->exp > p2->exp)      { append(&res, p1->coeff, p1->exp); p1 = p1->next; }
    else if (p1->exp < p2->exp) { append(&res, p2->coeff, p2->exp); p2 = p2->next; }
    else { append(&res, p1->coeff + p2->coeff, p1->exp); p1 = p1->next; p2 = p2->next; }
}
while (p1 != NULL) { append(&res, p1->coeff, p1->exp); p1 = p1->next; }
while (p2 != NULL) { append(&res, p2->coeff, p2->exp); p2 = p2->next; }
```

Polynomial multiplication (idea): multiply every term of P1 with every term of P2 (multiply coefficients, add exponents) and then add the like terms.

7. Quick Revision - CO1

Remember these points

- Data structure = way of organising data. Linear: array, linked list, stack, queue. Non-linear: tree, graph.
- Pointer stores an address. * gives value, & gives address. malloc() and free() manage dynamic memory.
- Array address: Base + i x size. Insertion needs right shift, deletion needs left shift.
- String = character array ending with '\0'.
- Linked list node = data + next. Insert at beginning: newnode->next = head ; head = newnode.
- Delete from beginning: temp = head ; head = head->next ; free(temp).
- Types: singly, doubly, header, circular singly, circular doubly.
- Polynomial node = coeff, exp, next. Add terms with equal exponents.

Practice questions

1. Define data structure. Explain the classification of data structures with examples.
2. Write short notes on pointers and dynamic memory allocation.
3. Explain insertion and deletion in an array with an algorithm and example.

4. Compare array and linked list.
5. Explain different types of linked lists with diagrams.
6. Write algorithms for insertion and deletion at the beginning and end of a singly linked list.
7. Explain how a polynomial is represented using a linked list and add two polynomials.